

Algoritmos y Estructuras de Datos I
CE1103
Escuela ingeniería en Computadores



Documento de explicación de estructuras de
datos utilizadas
Proyecto #1 Monopoly TEC

Integrantes del grupo:

Francisco Li
Julian Sanabria
Esteban Sanchez
Andrew Barquero

Profesor:
Luis Barboza

Semestre II
2026

En el presente documento se explican las estructuras de datos lineales implementadas por el equipo para el desarrollo del Proyecto 1: Monopoly Distribuido con Estructuras Lineales, correspondiente al curso Algoritmos y Estructuras de Datos I.

Para cada estructura se detalla:

- su propósito dentro del sistema
- el motivo de su elección frente a otras alternativas
- las operaciones principales que ofrece

Con el fin de evidenciar la correcta aplicación de los conceptos vistos en el curso dentro de una aplicación cliente-servidor real.

Nodo Doble

¿Que es?

Un nodo doble es una estructura que almacena un dato y dos referencias: una hacía el nodo siguiente y otra hacia el nodo anterior. Esto permite que los nodos estén conectados en ambas direcciones, por lo que se puede avanzar o retroceder dentro de una estructura de datos. En este nodo, el nodo es genérico, por lo que puede almacenar diferentes tipos de datos.

¿Para que se usó en el proyecto?

El nodo se utilizó como elemento base de todas las estructuras de datos: lista doblemente enlazada, lista circular doblemente enlazada y cola circular. Al encontrar a los nodos en cada estructura, son los encargados de almacenar la información de muchas clases, como pueden ser propiedades en la lista doblemente enlazada, casillas en la lista circular doblemente enlazada, jugadores en la cola circular.

¿Por qué un nodo doble y no simple?

Esto se debe a que una lista doblemente enlazada necesita acceder tanto al elemento siguiente como al anterior. Un nodo simple únicamente tendría una

referencia hacia el siguiente nodo, por lo que no permitiría recorrer la lista hacia atrás de manera directa. Al tener ambas referencias, se facilita el recorrido desde el inicio o desde el final de la lista. Como por ejemplo en el tablero, si no fueran nodos dobles, no se podría retroceder en el tablero, restringiendo algunas funciones o eventos especiales.

Principales operaciones

Método	Qué hace	Complejidad
NodoDoble(valor)	Crea un nuevo nodo e inicializa su valor, dejando las referencias anterior y siguiente en null.	O(1)
(get/set)Valor	Permite obtener o modificar el dato almacenado en el nodo.	O(1)
(get/set) Siguiente()	Permite obtener o modificar la referencia al siguiente nodo.	O(1)
(get/set)Anterior()	Permite obtener o modificar la referencia al nodo anterior.	O(1)

Lista Doblemente Enlazada

¿Qué es?

Una lista doblemente enlazada es una estructura lineal de datos compuesta por nodos dobles, donde cada nodo contiene un valor y referencias hacia el nodo anterior y el siguiente. La lista mantiene una cabeza o head, que corresponde al primer elemento, y una cola o tail, que corresponde al último. Esto permite recorrer la lista tanto desde el inicio hacia el final como desde el final hacia el inicio.

¿Para que se usó en el proyecto?

En el proyecto se utilizó para poder hacer una lista de nodos que contengan propiedad y que esta lista de propiedades fuera un atributo del jugador. También se utilizó para hacer una lista de transacciones y así la lista funciona como un historial y así poder recorrerla al final de la partida para imprimir el historial en el .txt.

¿Por qué una lista doblemente enlazada y no una lista enlazada simple?

Esto se debe a que el proyecto requiere recorrer los elementos tanto desde el inicio como desde el final. Una lista enlazada simple únicamente permite avanzar hacia el siguiente elemento, mientras que la lista doblemente enlazada permite avanzar y retroceder directamente mediante las referencias Siguiente y Anterior. Esto hace que operaciones como recorrer la estructura desde el final sean posibles sin tener que recorrer nuevamente todos los elementos desde la cabeza. Por ejemplo, no se podría imprimir el historial de más reciente a más antigua si no se puede recorrer de la tail hacia atrás.

Principales operaciones

Método	Qué hace	Complejidad
--------	----------	-------------

GetCantidad()	Devuelve la cantidad de elementos almacenados en la lista.	O(1)
AgregarAlFinal (Valor)	Agrega un nuevo elemento al final de la lista y actualiza las referencias correspondientes.	O(1)
RecorrerDesdeInicio()	Recorre la lista desde la cabeza hasta el último elemento.	O(n)
RecorrerDesdeFinal()	Recorre la lista desde la cola hasta el primer elemento.	O(n)
BuscarPor(predicado)	Recorre la lista y devuelve los elementos que cumplen con una condición determinada.	O(n)

Cola Circular

¿Que es?

Una cola circular es una estructura lineal de datos que se caracteriza por ser del tipo LIFO (Last in First Out), esto significa que el primer dato en haber entrado a la cola es el primero en salir. Por lo general en una cola simple el dato que está al frente desaparece de la cola al salir por ser el primero en la fila, pero como es una cola circular, el dato que sale se vuelve a incorporar a la cola y se convierte en el último elemento de la cola.

¿Para qué se usó en el proyecto?

Esta estructura de datos fue utilizada para crear la cola de los turnos de los jugadores, se crea una cola circular con los jugadores como elementos, cada vez que dentro del juego un jugador termina su turno, es sacado de la cola y vuelto a incorporar en la misma, por lo que se manda al final, creando así un sistema estable y justo de manejo de turnos.

¿Por qué circular y no una cola normal?

Esto es debido a que un sistema con una cola simple no sería capaz de administrar los turnos de los jugadores como lo hace su versión circular, la gracia de que sea circular es que se vuelva a encolar a cada jugador que sale, pero la cola simple no puede hacer eso naturalmente. Si se usara una cola simple se crearían bugs como que la partida se congele por que ya no hay nadie asignado a un turno.

Principales operaciones

Método	Qué hace	Complejidad
EstaVacia()	Indica si no hay jugadores en la cola	O(1)
Encolar(valor)	Agrega un nuevo jugador al final de la cola	O(1)
Actual()	Devuelve el jugador al que le toca el turno actual	O(1)
AvanzarTurno()	Pasa el turno al siguiente jugador (circular)	O(1)
EliminarActual()	Saca de la cola al jugador actual (ej. si queda eliminado de la partida)	O(1)

Lista Circular Doblemente Enlazada

¿Que es?

La lista circular doblemente enlazada es una forma de almacenamiento lineal de datos compuesta por nodos dobles que tienen acceso a un nodo anterior y posterior a ellos, estos nodos contienen información de algún tipo. La lista con una cabeza o head que es el primer elemento de la lista. Esta es recorrible tanto como hacia la derecha como hacia la izquierda. A diferencia de las listas no circulares, al llegar al último elemento y avanzar uno más, se llega al head, osea que el ciclo se repite.

¿Para qué se usó en el proyecto?

En el proyecto se utilizó para el manejo del tablero y de las casillas, el tablero como tal es una lista doblemente enlazada circular y sus 24 casillas son nodos que contienen la información de la casilla. El head de la lista es el punto de inicio, y al llegar a la casilla en la posición 24 y avanzar una más, se llega a la casilla 1.

También se utiliza para la creación del mazo de cartas de evento, al caer en una casilla de evento se toma al azar de una lista circular doblemente enlazada de cartas llamada mazo una carta, y al usarse se avanza al siguiente elemento de la lista, osea, a la siguiente carta del mazo y al llegar a la última, se devuelve a la primera.

¿Por qué circular y doblemente enlazada y no una lista enlazada simple?

Porque el tablero no tiene un final real después de la última casilla se vuelve a la primera. Una lista lineal normal obligaría a controlar manualmente ese salto cada vez. Igual con el mazo de cartas, se deben repetir infinitamente hasta que la partida termine.

Principales Operaciones

Método	Qué hace	Complejidad
AgregarAlFinal(valor)	Agrega un elemento al final, manteniendo la circularidad	$O(1)$
ObtenerNodoEnPosicion(indice)	Devuelve el nodo en una posición dada, calculada de forma circular	$O(n)$
ObtenerEnPosicion(indice)	Igual que el anterior, pero devuelve solo el valor	$O(n)$
Siguiente(nodo) / Anterior(nodo)	Devuelve el nodo siguiente o anterior a uno dado	$O(1)$
Recorrer()	Recorre la lista completa una sola vuelta	$O(n)$
TomarPrimeroYEnviarAlFinal()	Devuelve el primer elemento y lo manda al final (usado para el mazo de cartas)	$O(1)$

